

Treat the following questions:**Exercise 1: (11 pts)**

Consider the following class named *Product*:

```
public class Product {
    private String name;
    private int quantity;
    private String category;

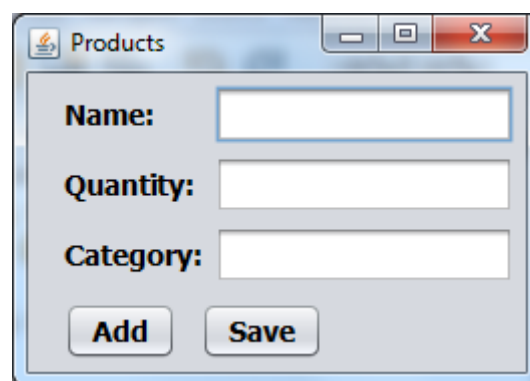
    public Product(String name, int quantity, String category) {
        this.name = name;
        this.quantity = quantity;
        this.category = category;
    }

    public String getName() {
        return name;
    }

    public int getQuantity() {
        return quantity;
    }

    public String getCategory() {
        return category;
    }
}
```

Consider the following graphical interface that allows the data input of a *Product* using text fields. The button *Add* allows the creation of a new *Product* object and adds it to an *ArrayList*. The button *Save* allows the saving of the objects *Product* of the *ArrayList* to a file. The file's data are loaded at program start-up using the method *load* below.



The corresponding code of this graphical interface is as follows:

```
public class ProductFrame extends Frame {
    private TextField name = new TextField(10);
    private TextField quantity = new TextField(10);
    private TextField category = new TextField(10);
    private Label lName = new Label("Name:");
    private Label lQuantity = new Label("Quantity:");
    private Label lCategory = new Label("Category:");
    private Button add = new Button("Add");
    private Button save = new Button("Save");
    private ArrayList<Product> productsList =
        new ArrayList<Product>();

    public ProductFrame() {
        // to be completed ... (1)
        this.pack();
        load();
        // to complete ... (2)
    }

    private void addPerformed() {
        // to be completed ... (3)
    }
}
```

```

}

private void savePerformed() {
    // to be completed ... (4)
}

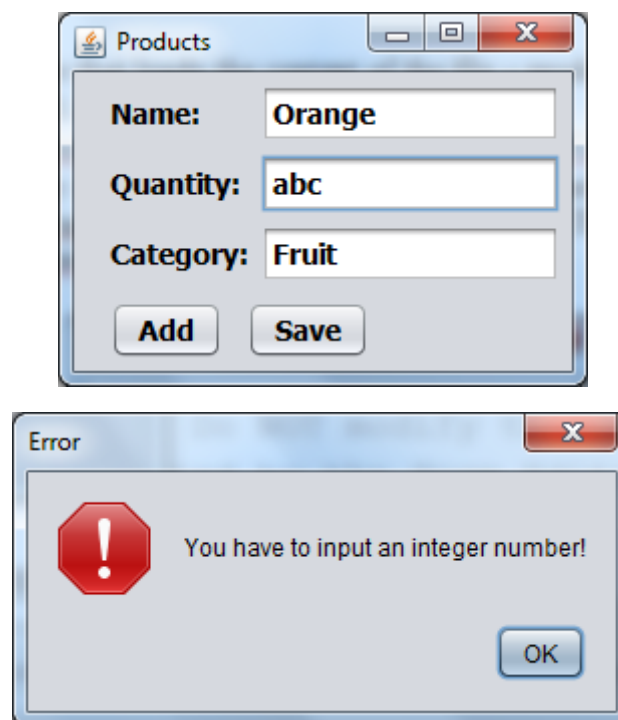
public void save2Performed() {
    // to be completed ... (5)
}

private void load() {
    // to be completed ... (6)
}
}

```

- Give in (1) the necessary code in order to obtain the above graphical interface. (You may use the layout managers *BorderLayout*, *FlowLayout*, and *GridLayout*).
- Give in (2) the code that adds an *ActionListener* to the button *Add* and executes the method *addPerformed*.
- Give in (3) the code that allows the data input in the text fields, to create an object *Product*, and to add it to the object *productsList*. The text fields should be empty after the addition.
- Give in (4) the code that opens the file « product.txt » and to write the content of the *productsList* to it. Each line must contain the data of one *Product* object as follows:

Orange,10,Fruit
- Give in (5) the code that serializes (persists) the *Product* objects of the *productsList* to a file « data.bin ». Which change should you do to the class *Product* regarding the persistence?
- Give in (6) the code that loads the content of the file « product.txt », creates the *Product* objects (one by line), and adds them to the *productsList*.
- The method *parseInt* raises an exception of type *NumberFormatException* if the String passed as parameter cannot be converted to an integer number. Change the code of the method *addPerformed()* in order to support this exception. The call of this exception will display a *JOptionPane* as follows:



Exercise 2: (5pts)

- Write the code that draws and fills a circle inside a square. The square's dimensions and its color are entered in the html code.
- Modify the previous code in order to display an error message if the square's parameters are greater than the applet's dimensions.

Exercise 3: (4pts)

Suppose that we have a database called « bookDB » with a username « hello » and a password « welcome ».

This database contains the table « Book » that is composed of two attributes: « code » and « title » with the following tuples (records):

Code	Title
JV	Java
DB	Database
OS	Operating System
DS	Data Structures

- Open a connection to the database, read the table's data and put them in an object of type *HashMap*.
- Use an iterator on the *HashMap* already created and then display each tuple on a line.